



UT1. MANEJO DE FICHEROS

ACTIVIDAD 8

Unmarshalling de archivo XML a objeto Java

Título: Deserialización de un archivo XML de usuarios con la API JAXB en Java.

Descripción: En esta actividad vamos a deserializar (unmarshal) un fichero XML que contiene un **usuario de un sistema de seguridad** y convertirlo en un objeto Java utilizando la API **JAXB**.

El objetivo es aprender a mapear un documento XML a una clase Java con anotaciones, de forma que podamos acceder a los datos como si fueran atributos de un objeto. Esta técnica es muy útil en el ámbito de la **gestión de usuarios y ciberseguridad**, donde los datos suelen almacenarse o intercambiarse en XML y necesitamos procesarlos en nuestra aplicación.

Objetivos:

- Comprender el proceso de **unmarshalling** para convertir un archivo XML en objetos Java.
- Utilizar la API JAXB para trabajar con datos XML de forma orientada a objetos.
- Acceder y mostrar la información deserializada (id, nombre, email, rol) desde un objeto Java.

Instrucciones:

1. Preparación del entorno:

- Crea un nuevo proyecto Java con Maven en tu IDE y llámalo **UnmarshalUsuario_JAXB**.
- Asegúrate de que el archivo usuario.xml esté disponible en el directorio del proyecto.

2. Definir la clase de destino para el XML (**se facilita esta clase**):

- Crea una clase Java llamada **Usuario** que represente la estructura de cada elemento "usuario" del XML.
- Utiliza las anotaciones de JAXB para mapear el archivo XML a la clase:
 - **@XmlRootElement** en la clase principal.
 - **@XmlAttribute** para mapear el atributo id.
 - **@XmlElement** en los campos correspondientes a cada elemento del XML, como nombre, email y rol.
- Asegúrate de definir constructores, getters y setters en la clase **Usuario** para cada campo.

3. Crear la clase principal:

- Crea una clase Java llamada **DeserializarUsuario** que contenga el método **main()**.



- Importa las clases necesarias de la API JAXB (jakarta.xml.bind.*).
- 4. Realizar el unmarshalling del archivo XML:**
- En el método main(), sigue estos pasos para deserializar el archivo XML:
 - Crea una instancia de JAXBContext utilizando la clase [Usuario](#).
 - Usa el método createUnmarshaller() para crear un objeto Unmarshaller.
 - Llama al método unmarshal() en el objeto Unmarshaller para deserializar el archivo usuario.xml.
- 5. Mostrar los datos en consola:**
- Una vez deserializado el archivo, muestra sus datos en la consola (id, nombre, email y rol). Recuerda acceder a cada atributo (id, nombre, email y rol) utilizando los métodos getter de la clase.
- 6. Verificación y pruebas:**
- Asegúrate de manejar excepciones como JAXBException para capturar cualquier error durante el unmarshalling.

Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Se presenta la tarea y el código es correcto, pero no se demuestra su ejecución con evidencias (capturas o logs de consola).
7	Se presenta la tarea, el código es correcto, y se demuestra el funcionamiento mediante capturas o salida de consola; sin embargo, no se describen los principales métodos y clases del programa.
9	Se completa la tarea correctamente en todos sus apartados, con código funcional, capturas de consola y breve descripción de métodos y clases.
10	Se añade alguna funcionalidad extra o detalle innovador que mejore la aplicación (por ejemplo, filtrar usuarios por rol, o mostrar estadísticas del número de administradores y usuarios).

Entrega:

- a. Incluye aquí el código fuente de las clases desarrolladas (Usuario.java y DeserializarUsuario.java):

CLASE Usuario.java :

```
import jakarta.xml.bind.JAXBContext;
import jakarta.xml.bind.JAXBException;
import jakarta.xml.bind.Unmarshaller;
import jakarta.xml.bind.annotation.XmlAttribute;
import jakarta.xml.bind.annotation.XmlElement;
import jakarta.xml.bind.annotation.XmlRootElement;
import java.io.File;
```



```
/**
 * @author Juan Gabriel Galarza Claros
 */

@XmlRootElement
public class Usuario {
    @XmlAttribute
    int id;
    String nombre;
    String email;
    String rol;

    public Usuario() {

    }

    public Usuario(int id, String nombre, String email, String rol) {
        this.id = id;
        this.nombre = nombre;
        this.email = email;
        this.rol = rol;
    }

    // Getters
    @XmlAttribute(name = "id")
    public int getId() {
        return id;
    }

    @XmlElement(name = "nombre")
    public String getNombre() {
        return nombre;
    }

    @XmlElement(name = "email")
    public String getEmail() {
        return email;
    }

    @XmlElement(name = "rol")
    public String getRol() {
        return rol;
    }

    // Setters
    public void setId(int id) {
        this.id = id;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public void setRol(String rol) {
```



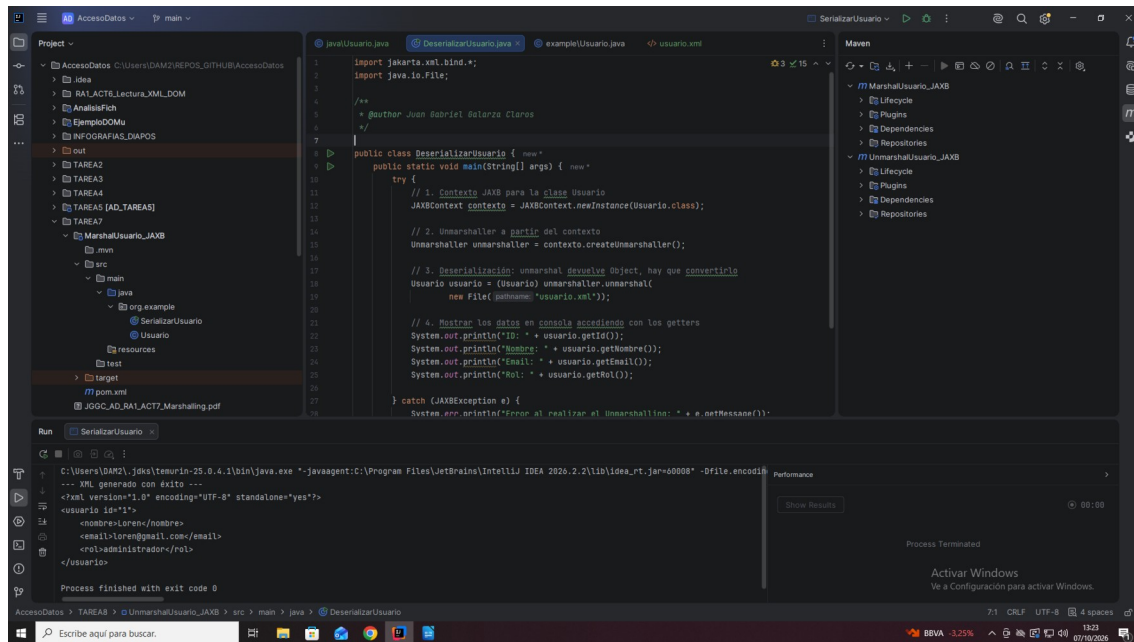
```
this.rol = rol;  
}  
}
```

CLASE DescentralizarUsuario.java :

```
import jakarta.xml.bind.*;  
import java.io.File;  
  
/**  
 * @author Juan Gabriel Galarza Claros  
 */  
  
public class DeserializarUsuario {  
    public static void main(String[] args) {  
        try {  
            // 1. Contexto JAXB para la clase Usuario  
            JAXBContext contexto = JAXBContext.newInstance(Usuario.class);  
  
            // 2. Unmarshaller a partir del contexto  
            Unmarshaller unmarshaller = contexto.createUnmarshaller();  
  
            // 3. Deserialización: unmarshal devuelve Object, hay que convertirlo  
            Usuario usuario = (Usuario) unmarshaller.unmarshal(  
                new File("usuario.xml"));  
  
            // 4. Mostrar los datos en consola accediendo con los getters  
            System.out.println("ID: " + usuario.getId());  
            System.out.println("Nombre: " + usuario.getNombre());  
            System.out.println("Email: " + usuario.getEmail());  
            System.out.println("Rol: " + usuario.getRol());  
  
        } catch (JAXBException e) {  
            System.err.println("Error al realizar el Unmarshalling: " + e.getMessage());  
        }  
    }  
}
```

- b. Incluye aquí captura/s de pantalla que demuestren la correcta ejecución del código (Y DONDE SEA VISIBLE LA DOCUMENTACIÓN DEL PROYECTO CON JAVADOC Y EL AUTOR DE LA ACTIVIDAD):

CAPTURA DE PANTALLA



c. Describe aquí las clases, métodos y variables principales utilizadas en el programa:

CLASE Usuario.java :

Clases principales

- Usuario: Clase modelo que representa los datos de un usuario y define la estructura del documento XML mediante anotaciones JAXB.
- JAXBContext: Punto de entrada de la librería JAXB que gestiona el contexto de enlace para la clase Usuario.
- Unmarshaller: Objeto encargado de deserializar el contenido XML y convertirlo en un objeto Java en memoria.

Métodos principales

- Usuario(): Constructor vacío obligatorio para que JAXB pueda instanciar la clase mediante reflexión.
- Usuario(id, nombre, email, rol): Constructor sobrecargado para inicializar todos los campos del objeto.
- getId(), getNombre(), getEmail(), getRol(): Métodos *getter* utilizados para obtener la información de cada campo y definir el mapeo de las etiquetas y atributos XML.
- setId(...), setNombre(...), etc.: Métodos *setter* que permiten asignar los valores leídos a las propiedades del objeto.

Variables principales

- id (int): Almacena el identificador único del usuario, mapeado como atributo XML.
- nombre (String): Contiene el nombre del usuario, mapeado como elemento hijo XML .
- email (String): Guarda la dirección de correo electrónico del usuario.
- rol (String): Define el rol o perfil del usuario dentro del sistema.

CLASE DescentralizarUsuario.java :**Clases principales**

- DeserializarUsuario: Clase ejecutable que contiene la lógica principal para leer y deserializar el archivo XML.
- JAXBContext: Clase de JAXB encargada de gestionar el contexto y la configuración para vincular la clase Usuario.
- Unmarshaller: Objeto que ejecuta el proceso de deserialización.
- File: Representa la ruta y el archivo físico del sistema (usuario.xml) que se va a leer.

Métodos principales

- main(String[] args): Punto de entrada de la aplicación donde se ejecuta todo el flujo de lectura y muestra de datos.
- JAXBContext.newInstance(...): Método estático que inicializa el contexto de JAXB para la clase indicada (Usuario.class).
- createUnmarshaller(): Método que genera la instancia del deserializador a partir del contexto.
- unmarshal(File): Lee el contenido XML del archivo y lo transforma en una instancia de tipo Object.
- getId(), getNombre(), getEmail(), getRol(): Métodos de la clase Usuario invocados para extraer los valores deserializados e imprimirlos en consola.

Variables principales

- contexto (JAXBContext): Almacena la configuración necesaria para trabajar con JAXB.
- unmarshaller (Unmarshaller): Contiene la herramienta que realiza la conversión de los datos XML.

- usuario (Usuario): Objeto resultante del casteo de la deserialización, que contiene los datos del archivo en memoria.
- e (JAXBException): Variable que captura cualquier error ocurrido durante el proceso de deserialización.